

**Module Code & Module Title****CS4051NI\CC4059NI Fundamental of Computing****70% Individual Coursework****Submission: Final Submission****Academic Semester: Summer Semester 2025****Credit: 15 credit semester long module****Student Name: Diya Maharjan****London Met ID: 24059043****College ID: NP01NT4S250025****Assignment Due Date: Wednesday, August 27, 2025****Assignment Submission Date: Wednesday, August 27, 2025****Submitted To: Aryan Thapa**

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

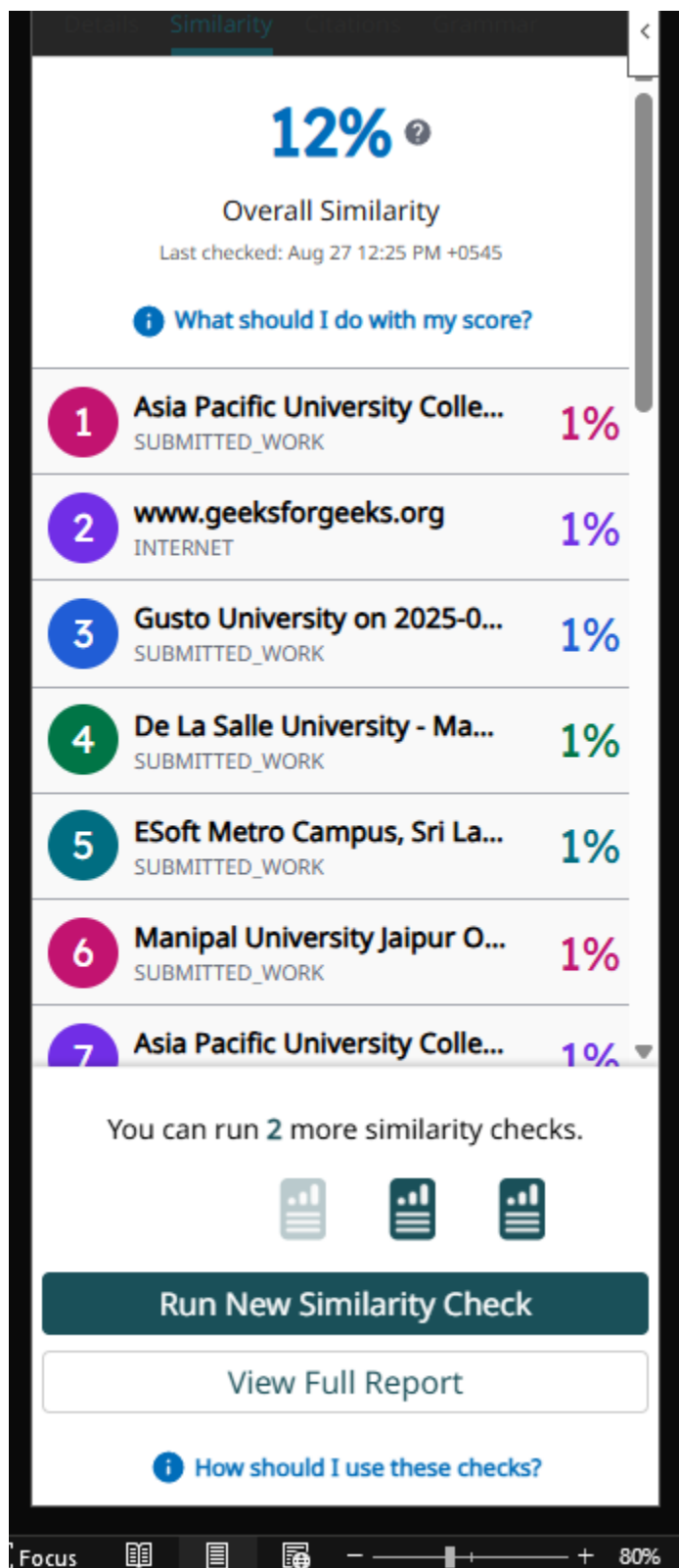


Table of Contents

1) Introduction.....	5
2) Aim and Objectives.....	5
2.1) Aim	5
2.2) Objectives.....	5
3) Technologies Used.....	6
3.1) Python.....	6
3.2) IDLE.....	7
3.3) Text file.....	7
3.4) draw.io.....	8
4) Algorithm.....	
5) Flowchart.....	
6) Pseudocode.....	
7) Data Structure.....	22
7.1) Types of Data structure.....	22
7.1.1) Primitive Data Structure.....	22
7.2) Complex Data Structure.....	23
7.3) The complex data structure that I used in my coursework.....	23
7.3.1) List.....	23
7.4) How did I use it? What impact did it have on my code?..	24
8) Testing.....	25
9) Conclusion.....	33

Table of Figures

1)Python.....	6
2)IDLE.....	7
3)Text files.....	7
4) Draw.io.....	8
5) Test 1.....	25
6) Test 1.....	25
7) Test 2.....	26
8) Test 2.....	26
9) Test 3.....	28
10) Test 3.....	28
11) Test 4.....	30
12) Test 4.....	30
13) Test 5.....	32
14) Test 5.....	32
15) Test 5.....	33

List of Table

1) Test 1.....	25
2) Test 2.....	26
3) Test 3.....	27
4 Test 4.....	29
5) Test 5.....	31

1) Introduction

Python is a high-level, programming language known for its simplicity, readability, and versatility. Python's syntax is clean and simple which makes it easy to read and write. It is high-level language and is also interpreted. Programmers use python for web development, cybersecurity and ethical hacking, Automation and scripting, Game development, etc.

This coursework is based on building a simple file handling system using Python to read, store, and display information about different types of shoes. The system reads data for a text file that contains details such as Shoe Type, Brand, Quantity, Price, and Origin, and then presents it in a clean, report-style format.

The focus of the project is to understand and implement file handling and basic data structures like list. The data is read from an input file (Information.txt) and stored using a list of list approach. This structure allows for organized storage and easier access to each shoe's information, which is later used to generate a report (shoes_report.txt).

2) Aims and objectives

2.1) Aim

To develop a file-handling system using Python to process and display shoe inventory data.

2.2) Objectives

- To read structured data from a text file using Python.
- To store data using lists for organized access.
- To extract details of the shoes.
- To display the shoe information in a readable and report-like format.
- To practice using basic data structures and file operations.

3) Technologies Used

3.1) Python

Python is a beginner friendly programming language that's easy to read, write and powerful enough to build anything from websites to apps, data analysis tools and even games. It is used by both beginners and professionals because of how simple the programming language is.

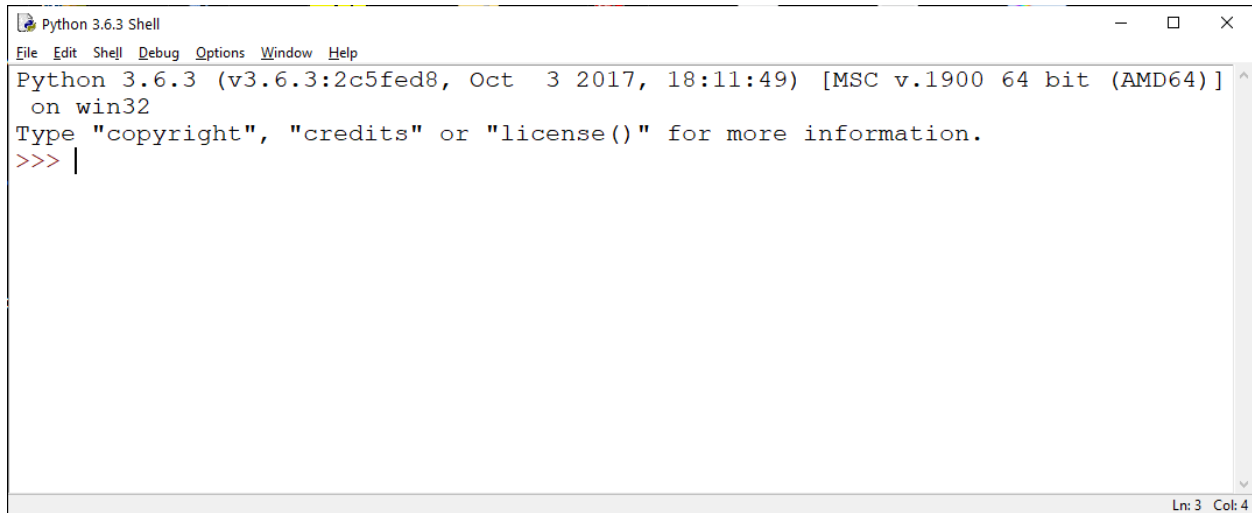


[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)

Figure 1:Python

3.2) Idle (Integrated Development and Learning Environment)

Idle is the built-in tool that comes with Python, where you can write, run, and test your Python code. It's designed to be super simple so beginners can easily learn and practice without needing any complicated setup.



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

Figure 2: Idle

3.3) Text file

A text file is a basic type of file that contains plain text. It does not contain any fancy formatting or codes. In programming like python, we use text files to store and manage data in a simple way.

```
Gate.H, Gate.T, Gate.Tdagger, Gate.S,  
Gate.Sdagger,  
  
Gate.X, Gate.Y, Gate.Z, Gate.eye
```

[This Photo](#) by Unknown Author is licensed under [CC BY](#)

Figure 3: Text file

3.4) Draw.io

Draw.io is a free browser-based diagramming tool that lets you create all kinds of diagrams, charts, and visuals. Unlike heavy software that requires installation, draw.io works directly in your browser and even integrates with platforms like Google Drive, GitHub. This makes it super handy for students, developers, or professionals who want to organize their ideas visually without paying for expensive software.

Using draw.io made it very easy for me to do my coursework as it has flowcharts, basic shapes, Network diagrams, integration symbols and many more. It was really helpful for my python's coursework for planning and presenting ideas.



[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)

Figure 4 : Draw.io

4) Algorithm

Step 1: Start

Step 2: Display the welcome message: SpeedWearz – Inventory Management System”

Step 3: Show main menu: Restock, Sell, Quit

Step 4: User Menu Selection

Step 5: Accept user input

Step 6: Validate input

Step 7: Read Inventory File

Step 8: Call read_file(“Information.txt”)

Step 9: Open file and read products records line by line

Step 10: Each record must have 6 fields: [ID, Name, Brand, Stock, Price, Origin]

Step 11: Store Products in a list (product_list)

Step 12: If file read fails, return an empty list

Step 13: Restock Process (if user selects 1)

Step 14: Display all current products using display_product()

Step 15: Ask for vendor details (name, phone). Validate phone is numeric.

Step 16: Repeat until vendor finishes restocking

Step 17: Ask for product ID. Search product in list.

Step 18: If not found, display error and retry.

Step 19: Show product details

Step 20: Ask how many items to add. Validate input (>0).

Step 21: Update stock of the product.

Step 22: Save restock entry into a restock_list.

Step 23: Ask if vendor wants to add more items. If yes, repeat.

Step 24: Build restock invoice with vendor details, table of product IDs, names and quantities added

Step 25: Display invoice on screen

Step 26: Ask for confirmation(y/n)

Step 27: Selling process (if user selects 2)

Step 28: Display all current products using display_product()

Step 29: Ask for customer details (name, phone). Validate phone.

Step 30: Repeat until customer finishes selection

Step 31: Ask for product ID. Search product in list

Step 32: If not found, show error and retry

Step 33: Show available stock

Step 34: Ask how many items to sell. Validate input

Step 35: If quantity exceeds stock, show error and retry.

Step 36: Deduct sold quantity from stock

Step 37: Save sale entry into a sell_list with product ID, name, quantity, price, origin.

Step 38: Ask if customer wants to add more items. If yes, repeat.

Step 39: Calculate totals

Step 40: Apply discount

Step 41: Build sale invoice with customer details products details, price details, Discount details

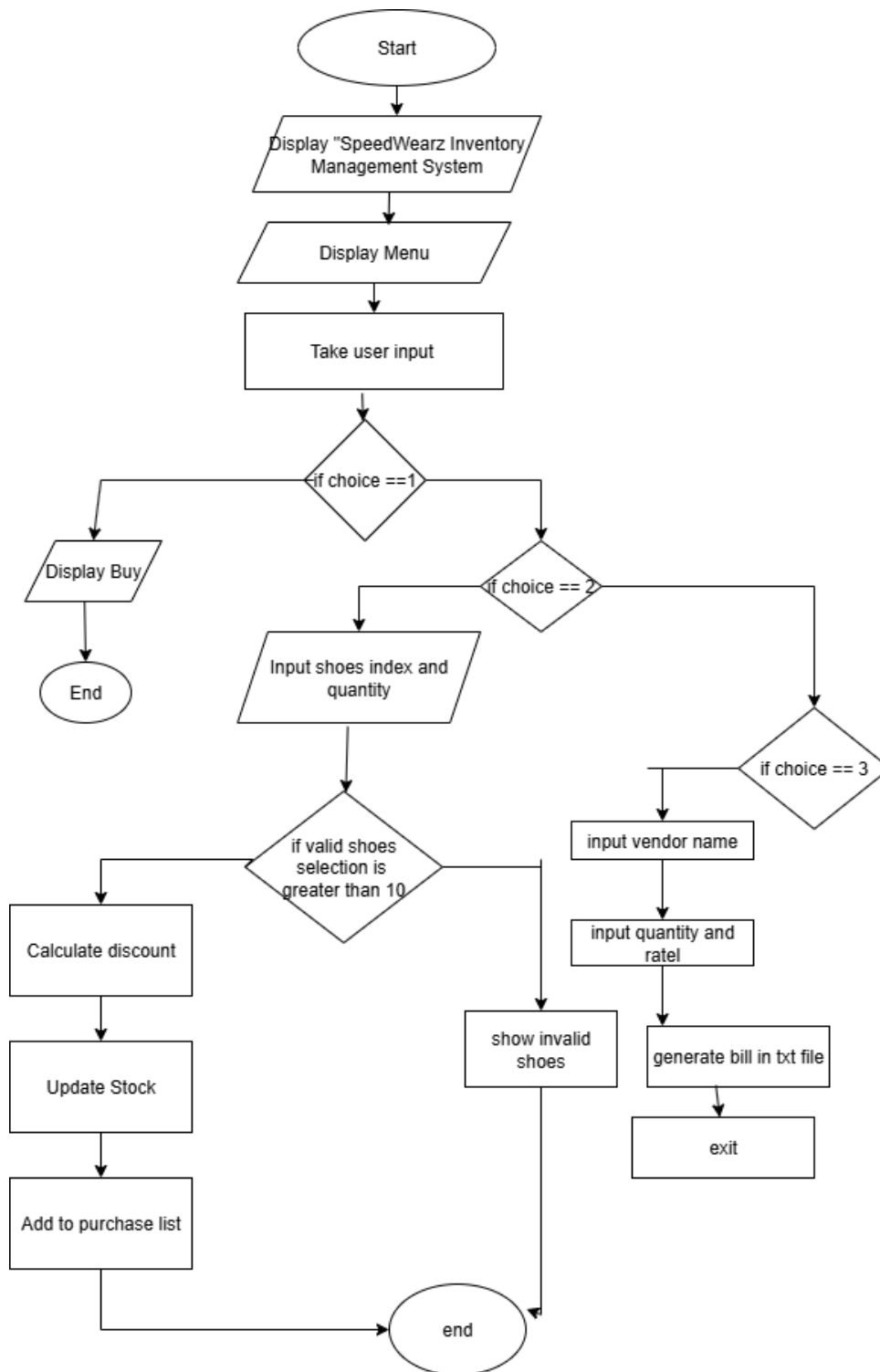
Step 42: Display invoice on screen

Step 43: Ask for confirmation (y/n)

Step 44: Exit the system

Step 45: End

7) Flow-chart



6) Pseudo Code:

Module Read

```
FUNCTION read_file(file_path):  
    TRY:  
        INITIALIZE product_list as empty list  
        OPEN file at file_path in read mode  
        LOOP through each line in file:  
            REMOVE newline from line  
            SPLIT line by comma → line_split  
            IF line_split length != 6:  
                CONTINUE  
            APPEND line_split to product_list  
        RETURN product_list  
    EXCEPT Exception:  
        PRINT "Error while reading file"  
        RETURN empty list
```

Module End

Module Write

```
FUNCTION write_file(file_path, product_list):  
    TRY:
```

```
OPEN file at file_path in write mode

LOOP through each product in product_list:

    CONVERT product items to string

    JOIN with commas

    ADD newline

    WRITE to file

EXCEPT Exception:

    PRINT "Error while writing file"
```

Module End

Module Operation

```
FUNCTION buy(product_list, file_path):

    PROMPT "Enter vendor name"

    LOOP UNTIL valid phone:

        TRY:

            PROMPT for vendor phone

            IF length != 10:

                PRINT "Phone number has to be 10 digits"

                CONTINUE

            CONVERT to integer

            BREAK

    EXCEPT ValueError:
```

PRINT "Phone number must be numbers only"

INITIALIZE buy_list as empty list

LOOP UNTIL user stops adding:

TRY:

PROMPT for product ID

FIND product by ID in product_list

IF not found:

PRINT "Product ID not found"

CONTINUE

DISPLAY product details

PROMPT for quantity

IF qty <= 0:

PRINT "Quantity must be greater than zero"

CONTINUE

CALCULATE line_total = price * qty

APPEND details to buy_list

UPDATE stock (increase)

PROMPT "Add more products? (y/n)"

```
IF not 'y': BREAK
```

```
EXCEPT ValueError:
```

```
PRINT "Please enter a valid number"
```

```
CREATE invoice_text with date, vendor, products, totals
```

```
PRINT invoice_text
```

```
PROMPT "Confirm buy? (y/n)"
```

```
IF 'y':
```

```
    CREATE file_name with timestamp
```

```
    WRITE invoice_text to text file
```

```
    CALL write_file(file_path, product_list)
```

```
    PRINT "Buy confirmed"
```

```
ELSE:
```

```
    PRINT "Buy cancelled"
```

```
FUNCTION sell(product_list, file_path):
```

```
    PROMPT "Enter customer name"
```

```
    LOOP UNTIL valid phone:
```

```
        TRY:
```

```
            PROMPT for phone
```

```
            IF length != 10:
```

```
                PRINT "Phone number has to be 10 digits"
```

CONTINUE

CONVERT to integer

BREAK

EXCEPT ValueError:

PRINT "Phone number must be numbers only"

INITIALIZE sell_list as empty list

LOOP UNTIL user stops adding:

TRY:

PROMPT for product ID

FIND product by ID

IF not found:

PRINT "Product ID not found"

CONTINUE

DISPLAY product details

PROMPT for quantity

IF qty > stock:

PRINT "Not enough stock"

CONTINUE

CALCULATE line_total

APPEND to sell_list

UPDATE stock (decrease)

PROMPT "Add more products? (y/n)"

IF not 'y': BREAK

EXCEPT ValueError:

PRINT "Please enter a valid number"

CALCULATE total_sum, discount, final_price

CREATE invoice_text with products, totals, discount

PRINT invoice_text

PROMPT "Confirm sale? (y/n)"

IF 'y':

CREATE file_name with timestamp

WRITE invoice_text to text file

CALL write_file(file_path, product_list)

PRINT "Sale confirmed"

ELSE:

PRINT "Sale cancelled"

Module End

Module Main

```
IMPORT read_file FROM read
```

```
IMPORT write_file FROM write
```

```
IMPORT buy, sell FROM operation
```

```
FUNCTION display_product(file_data):
```

```
    PRINT product list headers
```

```
    LOOP through each product in file_data:
```

```
        PRINT product details
```

```
FUNCTION main():
```

```
    PRINT "SpeedWearz Inventory Management System"
```

```
    PRINT menu options (1. Buy, 2. Sell, 3. Quit)
```

```
    LOOP FOREVER:
```

```
        PROMPT user for option
```

```
        TRY:
```

```
            CONVERT input to integer
```

```
            IF choice not in range 1–3:
```

```
                PRINT "The number has to be between 1 and 3"
```

```
            CONTINUE
```

```
        EXCEPT ValueError:
```

```
            PRINT "The input has to be a number"
```

CONTINUE

IF choice == 1:

 READ inventory FROM 'Information.txt'

 DISPLAY product list

 CALL buy(inventory, 'Information.txt')

ELSE IF choice == 2:

 READ inventory FROM 'Information.txt'

 DISPLAY product list

 CALL sell(inventory, 'Information.txt')

ELSE IF choice == 3:

 EXIT program

CALL main()

Module End

7) Data Structure

Data Structure are ways to store, organize and manage data so that we can use it efficiently in a program. A data structure decides how data is arranged in memory so operations like searching, adding, deleting and updating become faster and easier. Without proper data structures, computers would still work but they'd be painfully slow and messy when handling large amounts of information.

7.1) Types of Data Structures.

7.1.1) Primitive Data Structure:

These are the simplest forms of storing data, directly supported by programming languages. Examples include integers, floats, characters, and Booleans.

- **Integer** -It is a data type used to store whole numbers without any decimal parts. These numbers can be positive, negative, or zero, such as -100, 0, or 256. Integers are very common for everyday tasks like counting items, keeping track of scores, or indexing elements in a list. Depending on the programming language and system, integers usually take up 2 to 4 bytes of memory. There are also variations like short, long, or unsigned int which allow storing smaller or larger ranges of whole numbers.
- **Float**: Floating point numbers, often called float, and their more precise cousin double, are designed to store numbers with decimal points. These include values like 3.1416, -0.005, or 99.99. Floats typically use 4 bytes of memory, while doubles use 8 bytes, offering greater precision especially important in scientific or financial calculations. However, due to the way computers represent these numbers in binary, floating-point values can sometimes have tiny rounding errors.
- **Boolean**: Boolean data types store only two possible values: True or False. Though they often use a full byte of memory, they conceptually require just one bit. Booleans are essential in programming for making decisions — they act like simple yes/no or on/off switches, controlling the flow of logic in your code through conditions like if statements and loops.

- **String:** Strings are sequences of characters used to store text such as "Hello", "Nepal 2025", or "@OpenAI". Depending on the language, strings may be considered primitive or non-primitive types. Each character in a string typically takes 1 to 2 bytes, and the total size depends on the string's length. Strings are used everywhere — from storing names and messages to passwords and file paths.

7.2) Complex Data Structure

Complex data structures are built using combinations of basic data types and more advanced constructs to organize and manage data efficiently. Python provides several built-in complex data structures like lists, tuples, dictionaries, and sets which allows you to store collection of data with different characteristics.

- **List:** A list is an ordered, mutable collection of items, which can hold mixed data types and can be resized dynamically.
- **Tuple:** A tuple is like a list but immutable, meaning once created, its contents can't be changed.
- **Dictionary:** stores data in key-value pairs, allowing fast lookup by key, making it ideal for associative arrays or maps.
- **Set:** is an unordered collection of unique elements, useful when you need to store distinct items and perform operations like unions and intersections.

7.3) The Complex data structure that I used in my coursework:

7.3.1) List

A list in Python is a built-in data structure that lets you store multiple values in a single variable. It is like a container or a basket that keeps items in order. Each item can be accessed using its position called index, starting from 0.

Example:

```
Shoes_data =[
    [" Loafer Light", "Goldstar", "200", "1000", "domestic"],
    ["Inigo 732", "Hush puppies", "150", "2500", "international"]
]
```

7.4) How did I use it? What impact did it have on my code?

In my coursework, I used the list data structure to store and manage shoe-related data read from a text file. Each line from the file containing details of the shoe was split into parts and stored as a list. These individuals list were then stored inside a main list, creating a list of lists. This made the data organized ad simple to work with.

By using list, I was able to efficiently store multiple shoe entries in a structured format. I was able to loop through each entry to display details in a readable format.

Overall, using a list helped make the program both functional and flexible, which is very essential.

8) Testing

Test 1: Implementation of try, expect (providing invalid input)

Objective	Show implementation of try and expect
Action	Input invalid alphabet instead of numeric value
Expected Result	Prints “The input has to be numeric”
Actual Result	Printed “The input has to be number”
Conclusion	Test was successful

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
=====
Choose an option:
1. Buy
2. Sell
3. Quit Program
: a
The input has to be a number.
:

```

Figure 5 : Inserting alphabet

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
=====
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 1
=====
===== PRODUCT LIST =====
ID  Name      Brand   Stock  Price  Origin
-----
1   Loafer Light GoldStar 1000   1242   domestic
2   Inigo 732   Caliber 249   2800   domestic
3   Lite Racer Adidas   1300   7000   international
=====
Enter vendor name: |

```

Figure 6 : After inserting number

Test 2: Invalid Product ID & Negative Quantity

Objective	Selection of purchase and sale of products
Action	Input negative value in the field Input non-existent value
Expected Result	Program does not stop when invalid values are entered and shows error message
Actual Result	Program did not stop when invalid values are entered and shows error message
Conclusion	Test successful

```

Python 3.13.5 (tags/v9.13.5:6b20a2, Jun 11 2025, 14:15:44) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 1

===== PRODUCT LIST =====
ID   Name      Brand      Stock  Price  Origin
-----
1   Loafer Light GoldStar    1242   1000   domestic
2   Inigo 732   Caliber    249   2800   domestic
3   Lite Racer  Adidas     1300   7000   international
-----

Enter vendor name: diya
Enter customer phone number: 1234567890

Enter product ID to buy: 4
Product ID not found.

Enter product ID to buy: |

```

Figure 7 : Invalid Product Id

```

Python 3.13.5 (tags/v9.13.5:6b20a2, Jun 11 2025, 14:15:44) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 1

===== PRODUCT LIST =====
ID   Name      Brand      Stock  Price  Origin
-----
1   Loafer Light GoldStar    1242   1000   domestic
2   Inigo 732   Caliber    249   2800   domestic
3   Lite Racer  Adidas     1300   7000   international
-----

Enter vendor name: diya
Enter customer phone number: 1234567890

Enter product ID to buy: 2
Selected: Inigo 732 (Current Stock: 249, Price: Rs.2800)
How many to add?: -700
Quantity must be greater than zero.

Enter product ID to buy: |

```

Figure 8 : Negative quantity

Test 3: Purchasing multiple products and generating bill

Objective	Multiple purchase bill generation
Action	User select buy from menu Input the name and phone number of user's Enter product ID and quantity to buy Enter yes to purchase more Enter product Id and quantity to buy more Enter no to end the purchase Then, system generates a purchasing voice, saves detail on a text file and updates stock permanently.
Expected Result	Shell should display the bill, and the bill should be saved in separate text file.
Actual Result	The bill was displayed in the shell, and the bill was saved in separate text file.
Conclusion	Test successful.

```

Python 3.13.5 (tags/v3.13.5:4eb20a2, Jun 11 2025, 14:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97796\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 1

===== PRODUCT LIST =====
ID   Name      Brand      Stock  Price      Origin
-----
1   Loafer Light  SolidStar  1242    1000      domestic
2   Inigo 732    Caliber    249     2800      domestic
3   Lite Racer   Adidas     1300    7000      international
-----

Enter vendor name: diya
Enter customer phone number: 1234567890

Enter product ID to buy: 1
Selected: Loafer Light (Current Stock: 1242, Price: Rs.1000)
How many to add?: 100
New stock for Loafer Light: 1342
Add more products? (y/n): y

Enter product ID to buy: 2
Selected: Inigo 732 (Current Stock: 249, Price: Rs.2800)
How many to add?: 700
New stock for Inigo 732: 949
Add more products? (y/n): n

===== BUY INVOICE =====
Date   : 24/08/2025, 21:00:48
Vendor : diya
Phone  : 1234567890
-----
ID   Name      Price  Qty  Line Total
-----
1   Loafer Light  Rs.1000.00  100  Rs. 100000.00
2   Inigo 732    Rs.2800.00  700  Rs. 1960000.00
-----
TOTAL      Qty: 800      Total: Rs2060000.00
=====

Confirm buy? (y/n): y
Buy confirmed. Invoice saved as buy_invoice_24_08_2025-21-00-53.txt
=====

```

Figure 9 : Bill in IDLE

```

----- BUY INVOICE -----
Date   : 26/08/2025, 21:00:48
Vendor : diya
Phone  : 1234567890
-----
ID   Name      Price  Qty  Line Total
-----
1   Loafer Light  Rs.1000.00  100  Rs. 100000.00
2   Inigo 732    Rs.2800.00  700  Rs. 1960000.00
-----
TOTAL      Qty: 800      Total: Rs2060000.00
=====

```

Figure 10 : Buy Bill Invoice generated in Text file

Test 4: Selling multiple products and generating bill

Objective	Sales bill generation
Action	Select to sell to customers Input name and phone number of the customer Enter the product ID and quantity to sell Enter yes to sell more Enter the product ID and quantity to sell Enter no to stop selling
Expected Result	The bill is displayed in the shell, and the bill is saved in separate text file
Actual Result	The bill is displayed in the shell, and the bill is saved in separate text file
Conclusion	Test Successful



```

Python 3.13.5 (tags/v9.13.5:6b20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedHearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 2

----- PRODUCT LIST -----
ID  Name      Brand      Stock  Price  Origin
-----
1  Loafer Light GoldStar  1342   1000   domestic
2  Inigo 732    Caliber   949    2800   domestic
3  Lite Racer  Adidas    1300   7000   international

Enter customer name: diya
Enter customer phone number: 1234567890

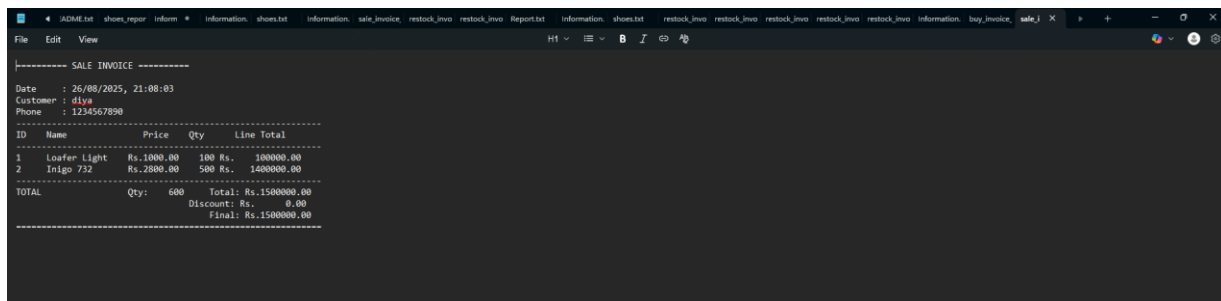
Enter product ID to sell: 1
Selected: Loafer Light (Available: 1342, Price: Rs.1000)
How many to sell?: 100
Add more products? (y/n): y

Enter product ID to sell: 2
Selected: Inigo 732 (Available: 949, Price: Rs.2800)
How many to sell?: 500
Add more products? (y/n): n

===== SALE INVOICE =====
Date       : 26/08/2025, 21:08:03
Customer   : diya
Phone      : 1234567890
-----
ID  Name      Price  Qty  Line Total
-----
1  Loafer Light  Rs.1000.00  100 Rs.  100000.00
2  Inigo 732    Rs.2800.00  500 Rs.  1400000.00
-----
TOTAL          Qty: 600      Total: Rs.1500000.00
                        Discount: Rs. 0.00
                        Final: Rs.1500000.00
=====
Confirm sale? (y/n): y
Sale confirmed. Invoice saved as sale_invoice_26_08_2025-21-08-04.txt
1

```

Figure 11 : Sell Bill in IDLE



```

----- SALE INVOICE -----
Date       : 26/08/2025, 21:08:03
Customer   : diya
Phone      : 1234567890
-----
ID  Name      Price  Qty  Line Total
-----
1  Loafer Light  Rs.1000.00  100 Rs.  100000.00
2  Inigo 732    Rs.2800.00  500 Rs.  1400000.00
-----
TOTAL          Qty: 600      Total: Rs.1500000.00
                        Discount: Rs. 0.00
                        Final: Rs.1500000.00
=====

```

Figure 12 : Sell Invoice generated in Text File

Test 5: Stock update in Information.txt (Text File)

Objective	Stock is updated when buying or selling products
Action	Enter to purchase from User Input the name and phone number of User Enter the product ID and quantity to buy Enter to purchase more Enter no to quit purchase Check Stock Re-run program Select to sell to customers Input name and phone number of the customer Select the product ID and quantity to sell Enter no to purchase more Check Stock
Expected Result	The quantity of products is updated in the text file when buying and selling products.
Actual Result	The quantity of products was updated in the text file when buying and selling products
Conclusion	Test successful.

```

===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 1

===== PRODUCT LIST =====
ID      Name      Brand      Stock Price      Origin
-----
1      Loafer Light  GoldStar   1242 1000    domestic
2      Inigo 732    Caliber    407  2800    domestic
3      Lite Racer   Adidas     1250 7000    international

Enter vendor name: diya
Enter customer phone number: 9874105263

Enter product ID to buy: 1
Selected: Loafer Light (Current Stock: 1242, Price: Rs.1000)
How many to add?: 100
New stock for Loafer Light: 1342
Add more products? (y/n): y

Enter product ID to buy: 3
Selected: Lite Racer (Current Stock: 1250, Price: Rs.7000)
How many to add?: 200
New stock for Lite Racer: 1450
Add more products? (y/n): n

===== BUY INVOICE =====
Date : 27/08/2025, 10:34:30
Vendor : diya
Phone : 9874105263
-----
ID      Name      Price      Qty      Line Total
-----
1      Loafer Light  Rs.1000.00  100 Rs.  100000.00
3      Lite Racer   Rs.7000.00  200 Rs.  140000.00
-----
TOTAL      Qty: 300      Total: Rs.150000.00

Confirm buy? (y/n): y
Buy confirmed. Invoice saved as buy_invoice_27_08_2025-10-34-38.txt
: |

```

Figure 13 : Quantity was added while purchasing the product

```

===== RESTART: C:\Users\97798\Downloads\python final\main.py =====
SpeedWearz
Inventory management system
-----
Choose an option:
1. Buy
2. Sell
3. Quit Program
: 2

===== PRODUCT LIST =====
ID      Name      Brand      Stock Price      Origin
-----
1      Loafer Light  GoldStar   1342 1000    domestic
2      Inigo 732    Caliber    407  2800    domestic
3      Lite Racer   Adidas     1450 7000    international

Enter customer name: diya
Enter customer phone number: 1234567890

Enter product ID to sell: 1
Selected: Loafer Light (Available: 1342, Price: Rs.1000)
How many to sell?: 100
Add more products? (y/n): y

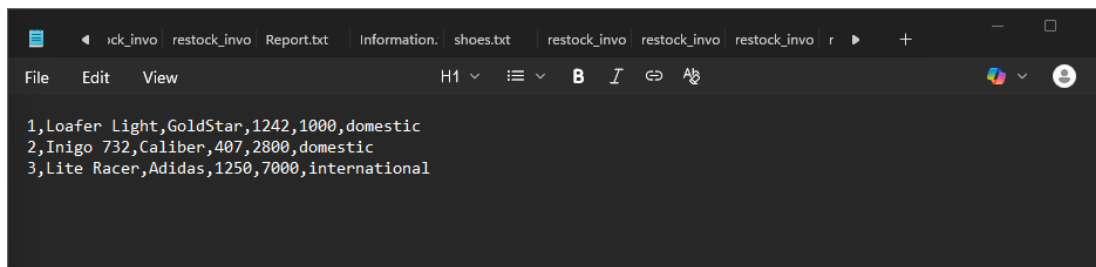
Enter product ID to sell: 3
Selected: Lite Racer (Available: 1450, Price: Rs.7000)
How many to sell?: 200
Add more products? (y/n): n

===== SALE INVOICE =====
Date : 27/08/2025, 10:38:48
Customer : diya
Phone : 1234567890
-----
ID      Name      Price      Qty      Line Total
-----
1      Loafer Light  Rs.1000.00  100 Rs.  100000.00
3      Lite Racer   Rs.7000.00  200 Rs.  140000.00
-----
TOTAL      Qty: 300      Total: Rs.150000.00
Discount: Rs. 0.00
Final: Rs.150000.00

Confirm sale? (y/n): y
Sale confirmed. Invoice saved as sale_invoice_27_08_2025-10-38-49.txt
: |

```

Figure 14 : Sale Bill generated



The screenshot shows a text editor window with multiple tabs. The active tab is 'shoes.txt'. The text in the editor is as follows:

```
1,Loafer Light,GoldStar,1242,1000,domestic  
2,Inigo 732,Caliber,407,2800,domestic  
3,Lite Racer,Adidas,1250,7000,international
```

Figure 15 : Quantity being reduced while selling product

9) Conclusion:

Developing the SpeedzWear Inventory Management System has provided an insightful and practical experience in understanding how modern businesses manage stock, sales, and customer interactions. The project successfully demonstrated how an automated system can streamline operations for a wholesale business dealing with both domestic and international products. By integrating features such as stock tracking, dynamic discount calculations, and invoice generation, the system ensures accurate record-keeping and efficient handling of sales and restocks.

The use of Python and its data structures, such as lists and dictionaries, proved effective in storing and manipulating product information. Modular programming allowed for a clear separation of responsibilities across different functions, making the program more readable, maintainable, and scalable. Handling errors and validating user input further enhanced the reliability of the application, ensuring that the system can manage unexpected scenarios without crashing or producing incorrect results.

Moreover, the ability to generate detailed invoices for both customers and vendors demonstrates the system's practical application in real-world business environments. Discounts for bulk purchases and incentives for domestic products not only reflect real business logic but also enhance the overall customer experience. The report and testing sections provide solid evidence of the system's functionality, confirming that it meets the requirements specified in the scenario.

In conclusion, this project not only strengthened technical skills in Python programming and algorithm design but also provided valuable insights into business operations, inventory management, and customer service processes. By automating routine tasks, the SpeedzWear Inventory Management System highlights the importance of technology in improving efficiency, accuracy, and profitability in wholesale operations. The knowledge gained through this project lays a strong foundation for developing more advanced inventory systems in the future, while also emphasizing the critical role of careful planning, testing, and documentation in software development.